ATLAS UNIVERSITY
Faculty of Engineering and Natural Science

C Programming Language

Systems Programming · Performance · Efficiency

Systems Level | High Performance | Portable | Hardware Access

Dr. Seyed-Sajad Ahmadpour
Department of Computer Engineering
seyed.ahmadpour@atlas.edu.tr

Prepared by: Hanieh Satari

Memory Management | Pointers | Structures

Introduction to C

The Mother of Modern Languages

Procedural Paradigm

Structured programming based on the concept of procedure calls. Focuses on steps to be executed rather than objects.

Close to Hardware

Provides low-level access to memory and system resources. Often called "portable assembly" for its efficiency.

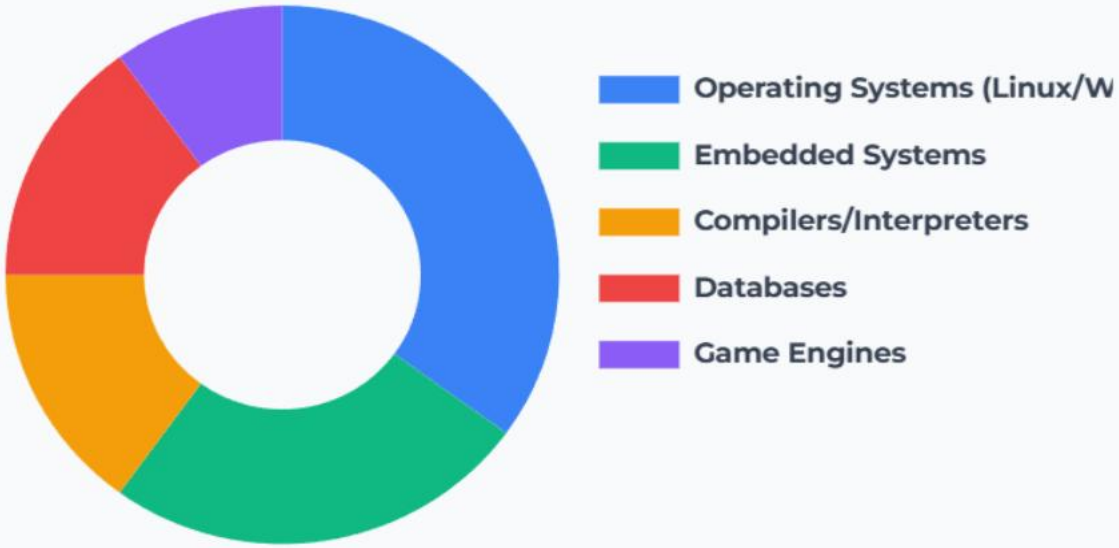
Minimalist Library

Small standard library (libc). Unlike Python or Java, C gives you raw building blocks, keeping the runtime footprint tiny.

Deterministic Performance

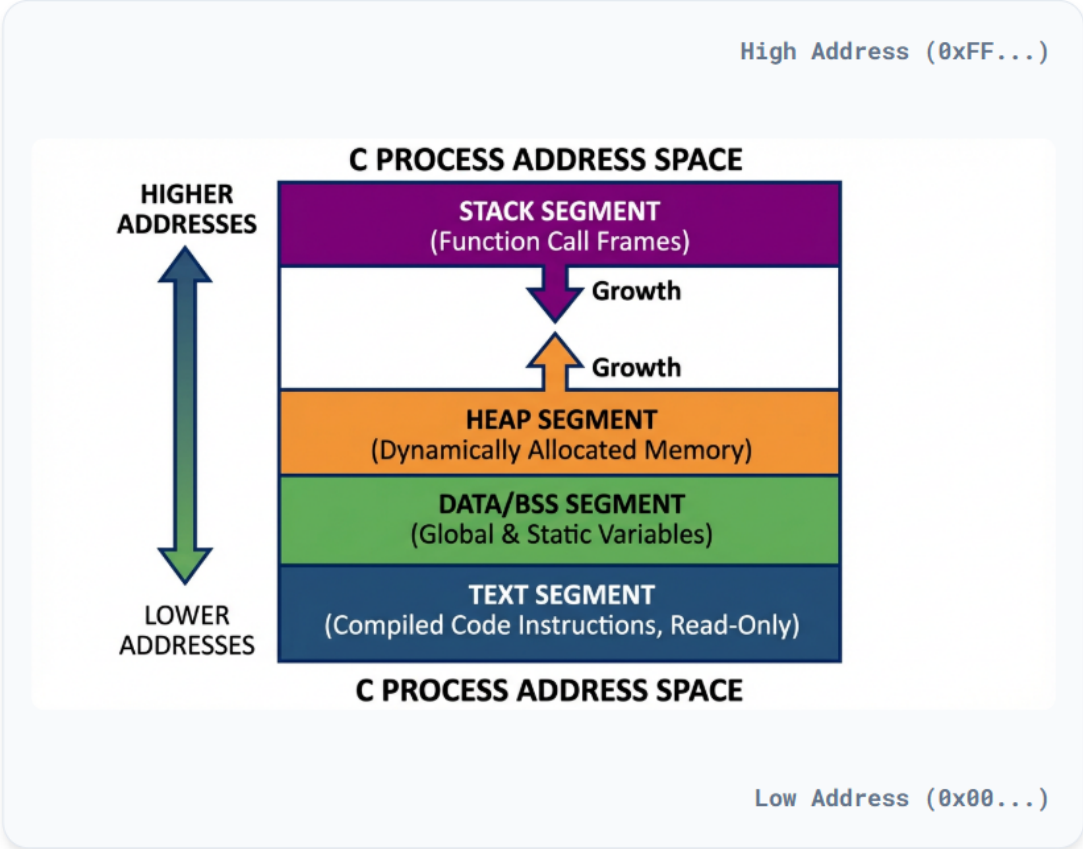
No hidden garbage collection pauses. You control exactly when memory is allocated and freed.

Where C is King



C Memory Model

The Anatomy of a Process Address Space



Stack Segment

Stores automatic variables, function parameters, and return addresses. Grows downwards from high memory. Managed automatically by the CPU.

Heap Segment

Dynamic memory allocated at runtime via `malloc()`, `calloc()`, etc. Grows upwards. Must be manually managed to avoid leaks.

Data & BSS

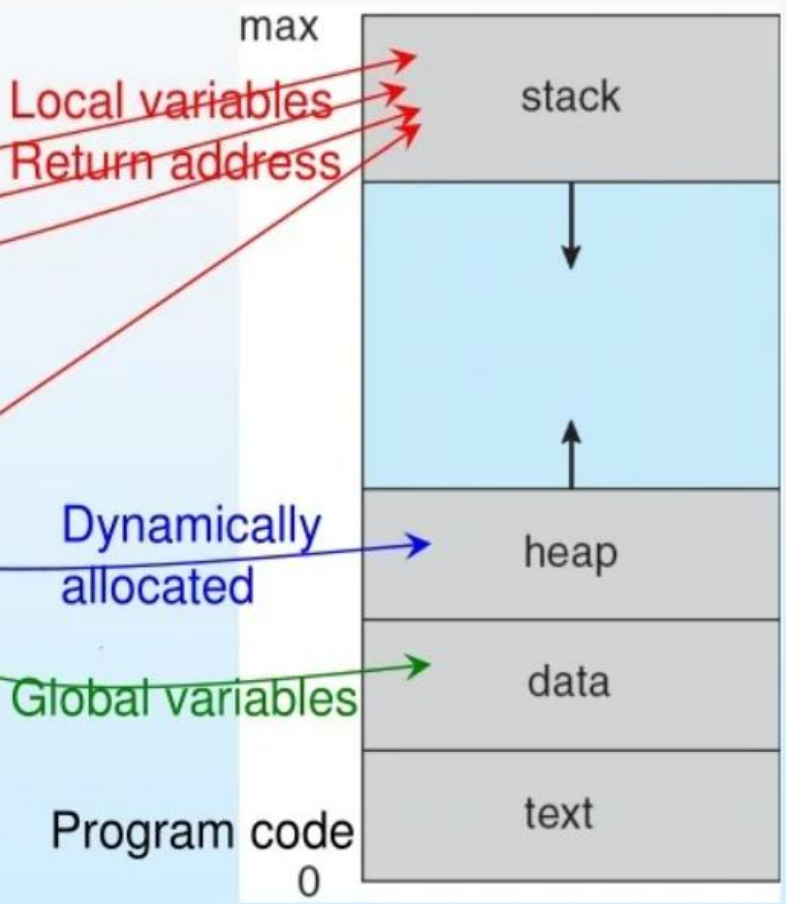
Data: Initialized global and static variables.
BSS: Uninitialized global/static variables (zero-initialized).

Text Segment

Contains the compiled machine code (instructions). Typically read-only to prevent accidental modification.

Process in Memory

```
int global = 0;
int main (int arg)
{
        float  local;
        char *ptr;

        ptr = malloc(100);
        local = 0;
        local += 10*5;
        .....
        ....
        foo();
        .... /* return addr */
        ....
        return 0;
}
```



Text vs. Data Segments

Text Segment

Code / Instructions

```
// Compiled Machine Code
void main() {
  printf("Hello");
}
```

- Contains compiled executable instructions
- **Read-Only** to prevent accidental modification
- Often sharable between processes

R-X

00101101 11001010
INSTRUCTIONS

VS

Data Segments

Global / Static Variables

```
int global_x = 10; // Data
static int y; // BSS (Zero)
```

- **Initialized Data:** Globals with values
- **BSS:** Uninitialized (auto-zeroed)
- **Read-Write** access during runtime

RW-

Initialized (x=10)

RW-

BSS (y=0)

Pointers: The Core Idea

Understanding Indirection and Address Manipulation

Stores Memory Addresses

A pointer is a variable that holds the memory address of another variable. It "points" to where data lives rather than holding the data itself.

Address-of Operator (&)

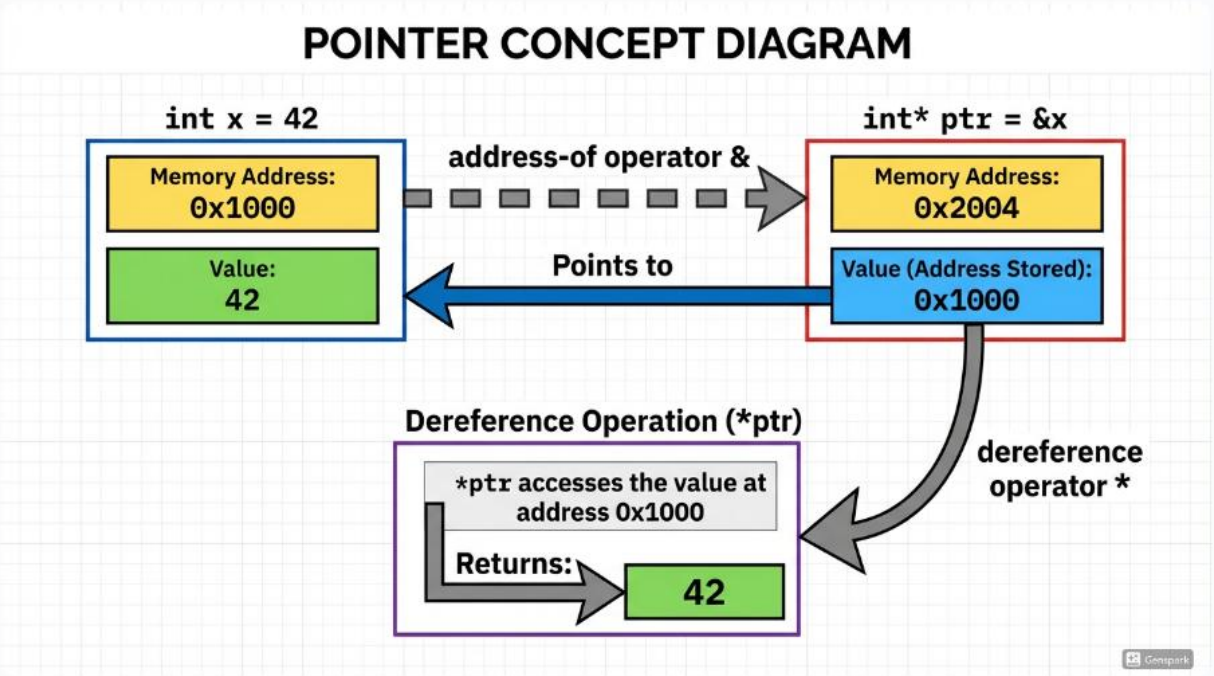
Used to retrieve the memory address of a variable. For example, `&x` gives you the location of `x` in memory.

Dereference Operator (*)

Used to access the value stored at the pointer's address. `*ptr` reads or modifies the value of the variable it points to.

Enables Dynamic Structures

Pointers are essential for dynamic memory allocation, arrays, and complex data structures like linked lists and trees.



Pointer States

Understanding the lifecycle and validity of memory references

Valid

```
int x = 10;
int *ptr = &x
```

- Points to a live, allocated object in memory.
- Safe to dereference (*ptr) to access value.

NULL

```
int *ptr = NULL;
if (ptr == NULL) ...
```

- Explicitly points to "nothing" or address 0.
- Safe to compare, but **crash** if dereferenced.

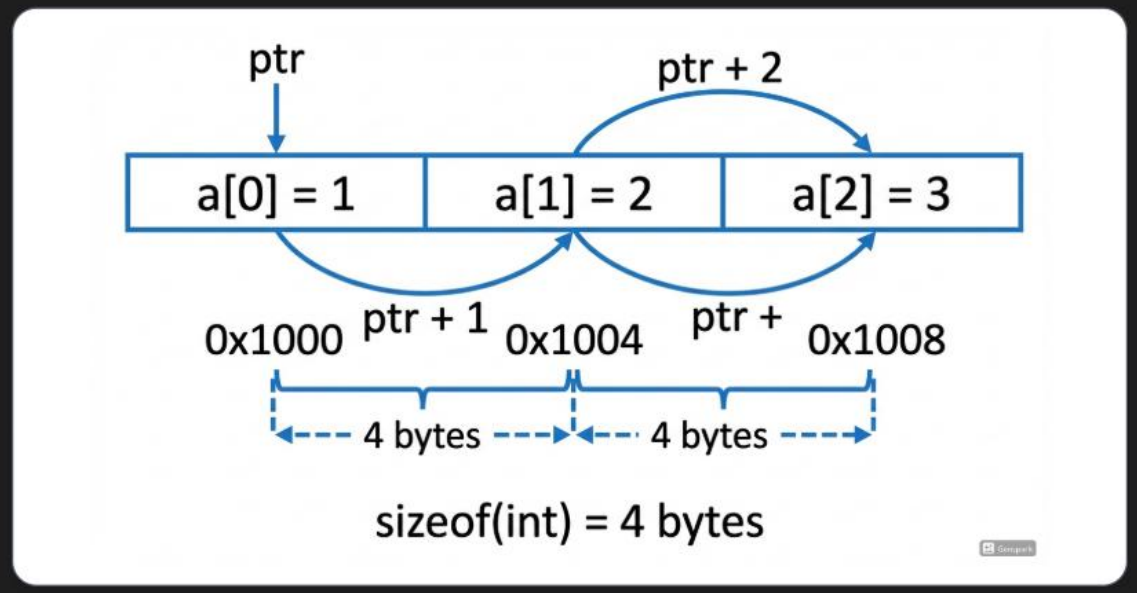
Invalid

```
int *ptr; // Uninit
free(ptr); // Dangling
```

- Uninitialized or points to freed memory (Dangling).
- **Undefined Behavior** if dereferenced.

Pointer Arithmetic

Navigating memory by data type size



- Moves in units of **sizeof(T)**, not just bytes
- `ptr + 1` advances to the immediate next element
- Defined only within the same array boundaries
- Subtraction returns number of elements between pointers

arithmetic.c

```c
int nums[3] = {10, 20, 30};
int *ptr = nums;  // Points to 10 (index 0)

printf("%d\n", *ptr);    // Prints 10

ptr++;                   // Moves 4 bytes forward

printf("%d\n", *ptr);    // Prints 20

// Pointer Subtraction
int *end = &nums[2];
long diff = end - ptr;
printf("Diff: %ld", diff); // Prints 1
```

Dynamic Memory: Overview

The Lifecycle of Heap Memory

1. Request
Determine needed size in bytes

→

2. Allocate
OS reserves block on Heap

→

3. Use
Read/Write data via pointer

→

4. Resize (Opt)
Expand or shrink block if needed

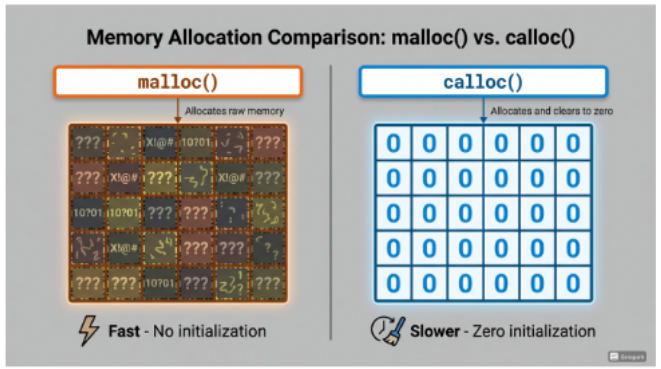
→

5. Free
Release memory back to OS

- Allocated on Heap at runtime using `malloc`
- Must handle allocation failures (NULL check)
- Crucial to `free()` exactly once to avoid leaks

malloc() Function

Allocating uninitialized memory blocks on the heap



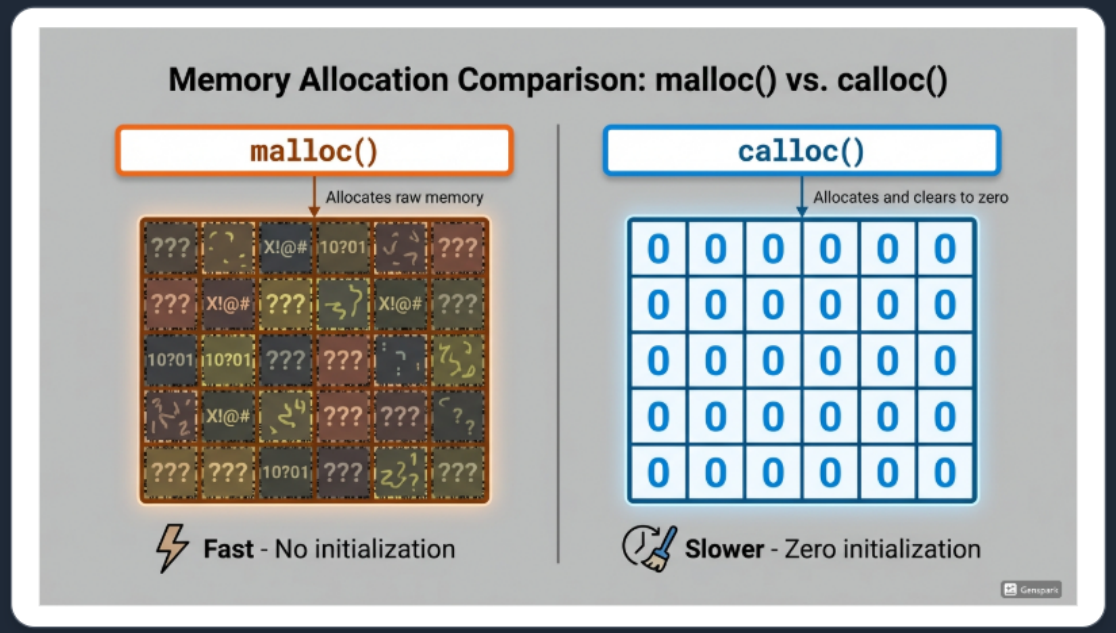
- **Memory Allocation:** Allocates a single large block of contiguous memory
- **Uninitialized:** Content is random "garbage" values, not cleared to zero
- **Return Value:** Returns `void*` pointer to first byte, or `NULL` if failed
- **Performance:** Generally faster than calloc() since it skips initialization

allocation.c

```c
#include <stdlib.h>

int *ptr;
int n = 5;

// Allocate memory for n integers

ptr = (int*) malloc(n * sizeof(int));


// Always check for NULL return!
if (ptr == NULL) {
 printf("Memory not allocated.\n");
 exit(0);
}
```

calloc() Function

Contiguous Allocation with Zero Initialization




```
void* calloc(size_t n, size_t size);
```

- Allocates space for **n** elements, each of **size** bytes.
- **Key Feature:** Automatically initializes all bits to zero.
- Prevents "garbage value" bugs common with `malloc()`.

zero_init.c

```c
#include <stdlib.h>
#include <stdio.h>

int main() {
 int n = 5;

 // Allocate and zero-initialize array

 int *ptr = (int*)calloc(n, sizeof(int));

 if (ptr == NULL) return 1;

 // Prints: 0 0 0 0 0
 for(int i=0; i<n; i++) {
 printf("%d ", ptr[i]);
    }
```

malloc() vs. calloc() Comparison

malloc()

Memory Allocation

```
int* ptr = (int*)malloc(n * sizeof(int));
```

- **Arguments:** Takes 1 argument (total size in bytes).
- **Initialization:** Does NOT initialize memory. Contains garbage values.
- **Performance:** Faster (skips zeroing out memory).

Uninitialized (Garbage)

VS

calloc()

Contiguous Allocation

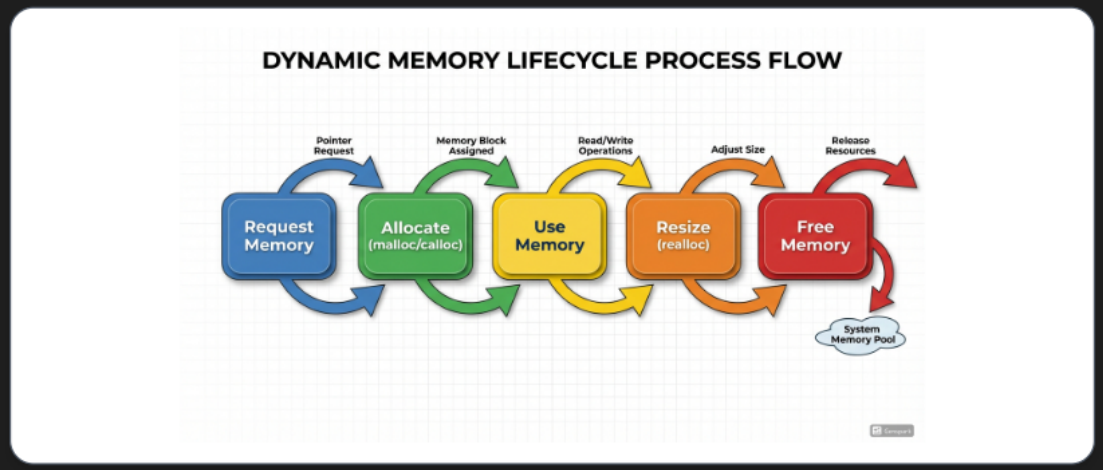
```
int* ptr = (int*)calloc(n, sizeof(int));
```

- **Arguments:** Takes 2 arguments (number of elements, element size).
- **Initialization:** Initializes all bits to zero.
- **Safety:** Safer (prevents uninitialized memory bugs), but slightly slower.

Zero Initialized

realloc() and free()

Managing memory lifecycle: Resizing and Deallocation



- **realloc(ptr, size)**: Resizes memory block. If moved, old data is copied.
- **free(ptr)**: Releases memory back to the heap. Crucial to prevent leaks.
- Always set pointer to **NULL** after freeing to avoid dangling pointers.
- Check `realloc` return value in a temporary pointer first.

lifecycle.c

```c
// 1. Allocate initial memory
int *arr = malloc(2 * sizeof(int));

// 2. Resize memory (Safe practice)
int *temp = realloc(arr, 4 * sizeof(int));

if (temp == NULL) {
 // Handle error, arr is still valid
} else {
 arr = temp; // Update pointer on success
}

// 3. Free memory when done
free(arr);

arr = NULL; // Prevent dangling pointer
```

C Structures: Intro

Organizing Data with User-Defined Types

User-Defined Aggregate Type

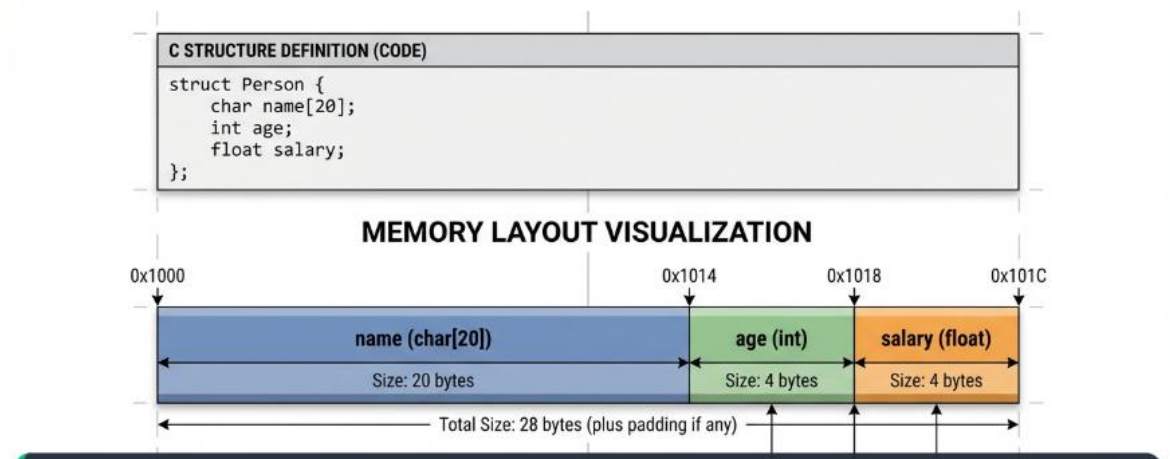
Structures allow you to group variables of different data types under a single name. Unlike arrays, which store elements of the same type.

Mixed Data Types

A single struct can contain integers, floats, characters, arrays, and even other structures. Ideal for representing complex objects like a 'Person' or 'Pixel'.

Accessing Members

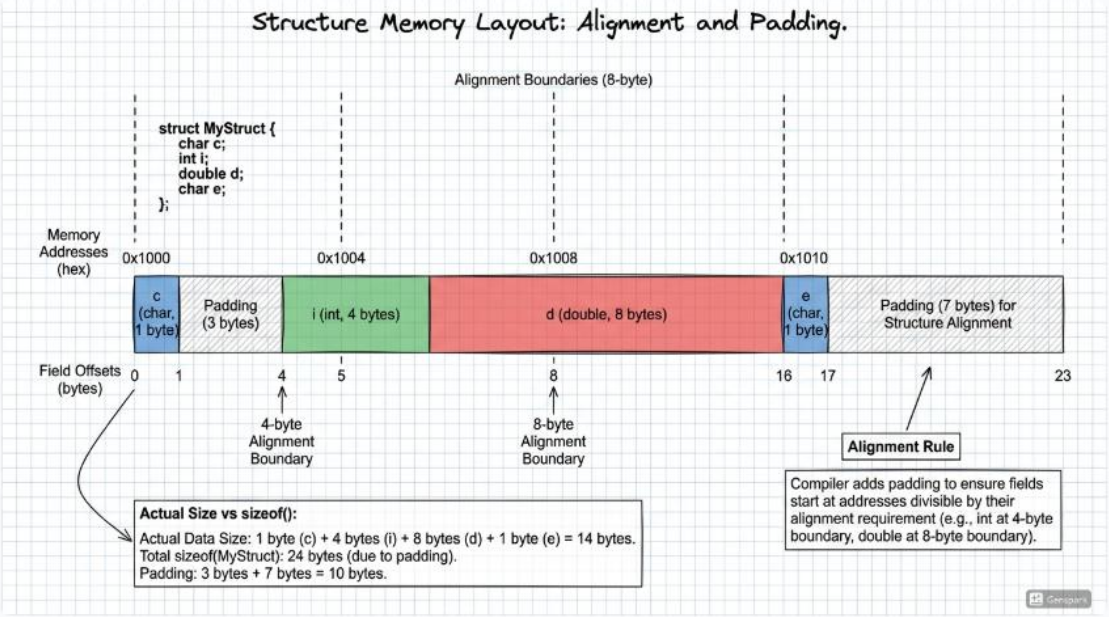
Use the dot operator (`.`) for direct structure variables. Use the arrow operator (`->`) when accessing via a pointer.




```c
struct Point {
  int x; // Member 1
  int y; // Member 2
};

struct Point p1 = {10, 20};
p1.x = 5; // Dot access
```

Structure Memory Layout

Alignment, Padding, and Memory Efficiency



Memory Alignment

CPUs access memory in chunks (words). Data types must be aligned to specific addresses (e.g., 4-byte integers at addresses divisible by 4) for efficient access.

Padding Bytes

The compiler automatically inserts unused "padding" bytes between members to satisfy alignment requirements. This "wasted" space increases the struct's size.

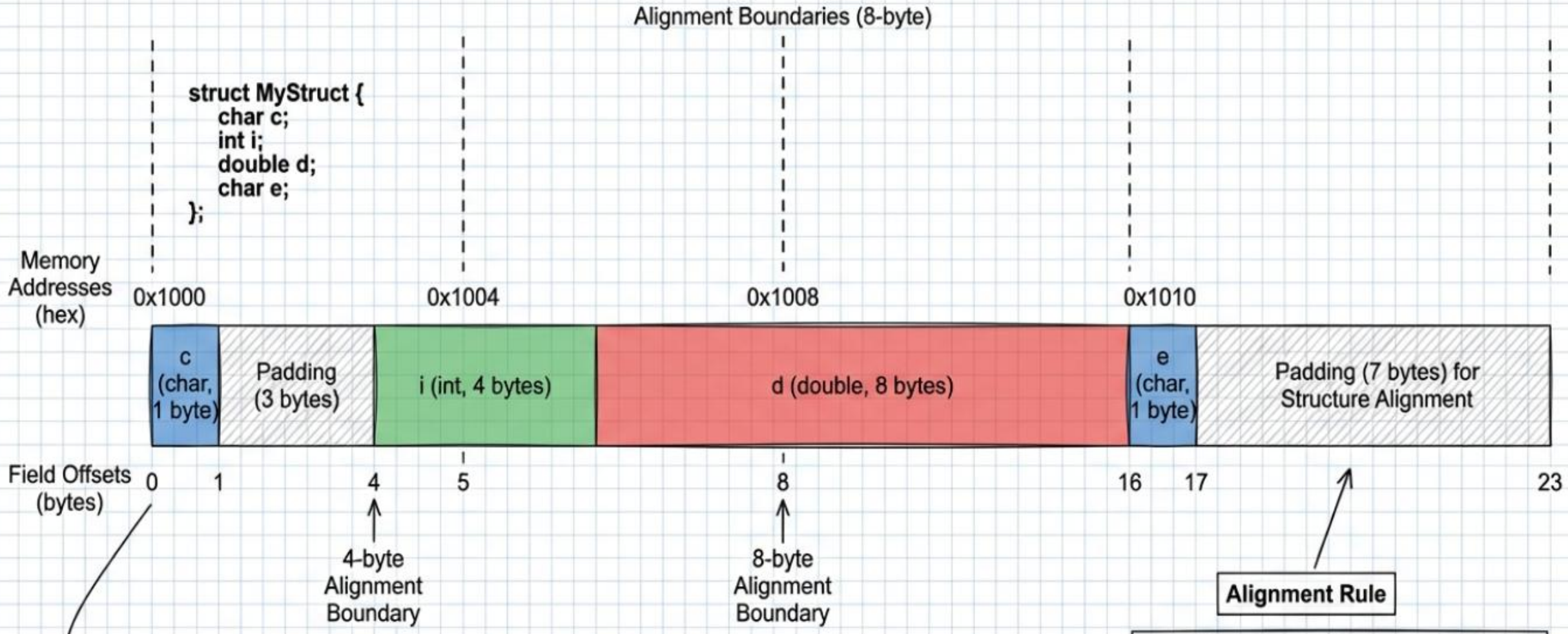
Actual Size vs. Sum

`sizeof(struct)` is often larger than the sum of its members. The total size is also padded to be a multiple of the largest member's alignment.

Optimization Tip

To minimize padding and save memory, order structure members by size, from largest to smallest (e.g., `double`, then `int`, then `char`).

Structure Memory Layout: Alignment and Padding.



Alignment Rule

Compiler adds padding to ensure fields start at addresses divisible by their alignment requirement (e.g., int at 4-byte boundary, double at 8-byte boundary).

Actual Size vs sizeof():

Actual Data Size: 1 byte (c) + 4 bytes (i) + 8 bytes (d) + 1 byte (e) = 14 bytes.
Total sizeof(MyStruct): 24 bytes (due to padding).
Padding: 3 bytes + 7 bytes = 10 bytes.

The Task: Write a program that asks the user how many students are in a class. Then, use malloc to create an array of Student structures. Input their names and marks, print them, and then—most importantly—clean up the memory.

THANK
YOU